MQTT-发布者例程

MQTT-发布者例程
例程讲解
简介
第一步：安装调试软件MQTTX
第二步：MQTT-发布者 启动
代码解析
1. 模块导入部分
2. 配置参数
3. `connect_wifi` 函数
4. 主程序部分
流程图

例程讲解

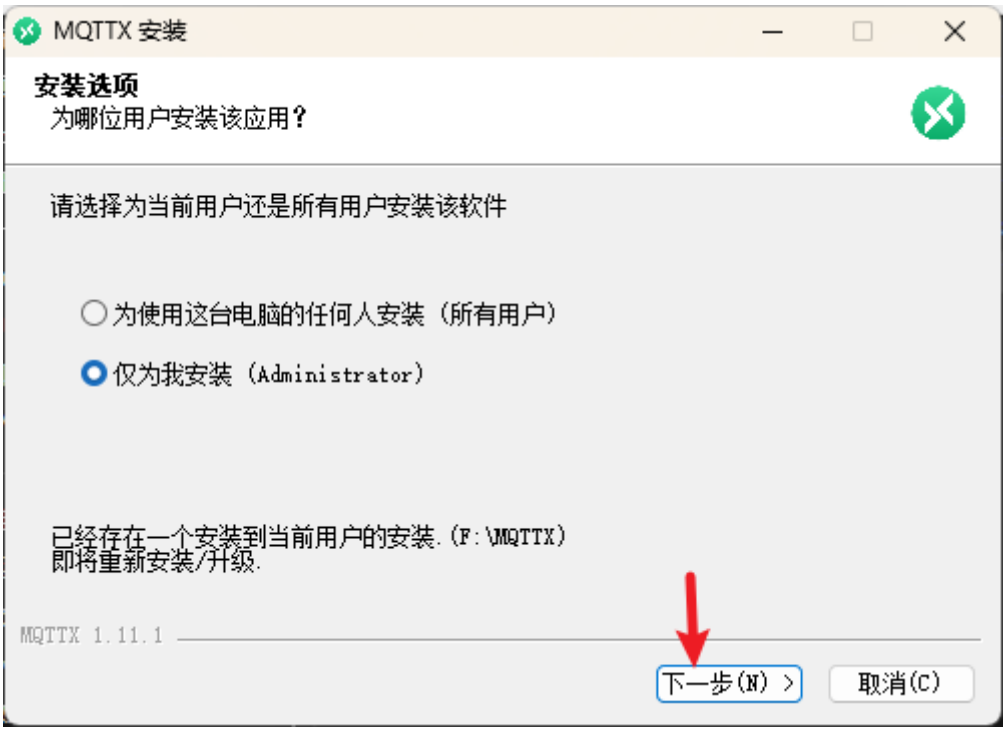
简介

本节我们使用K230实现MQTT发布者的功能。

关于MQTT的基础介绍，可以跳转到下面的 MQTT教程 部分查看

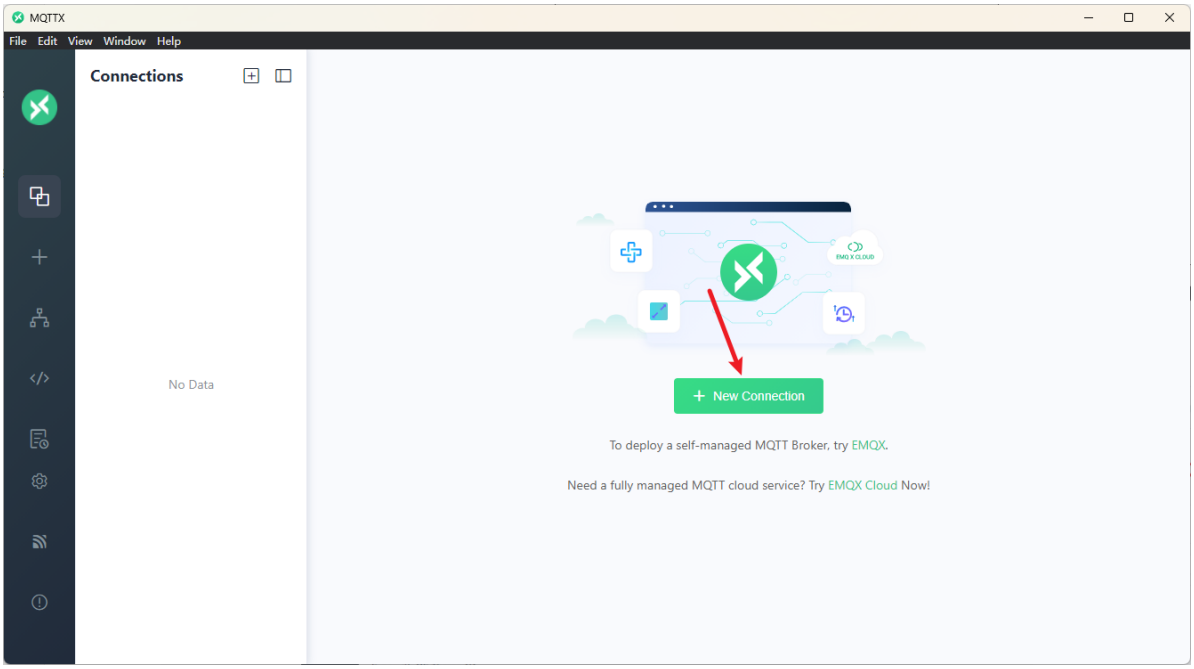
第一步：安装调试软件MQTTX

在运行例程之前，我们需要先安装一下MQTTX作为调试软件，大家可以在我们的配套资料中找到这个软件的安装包（Windows版本。其他系统版本可以在这个链接中找到MQTTX 下载）

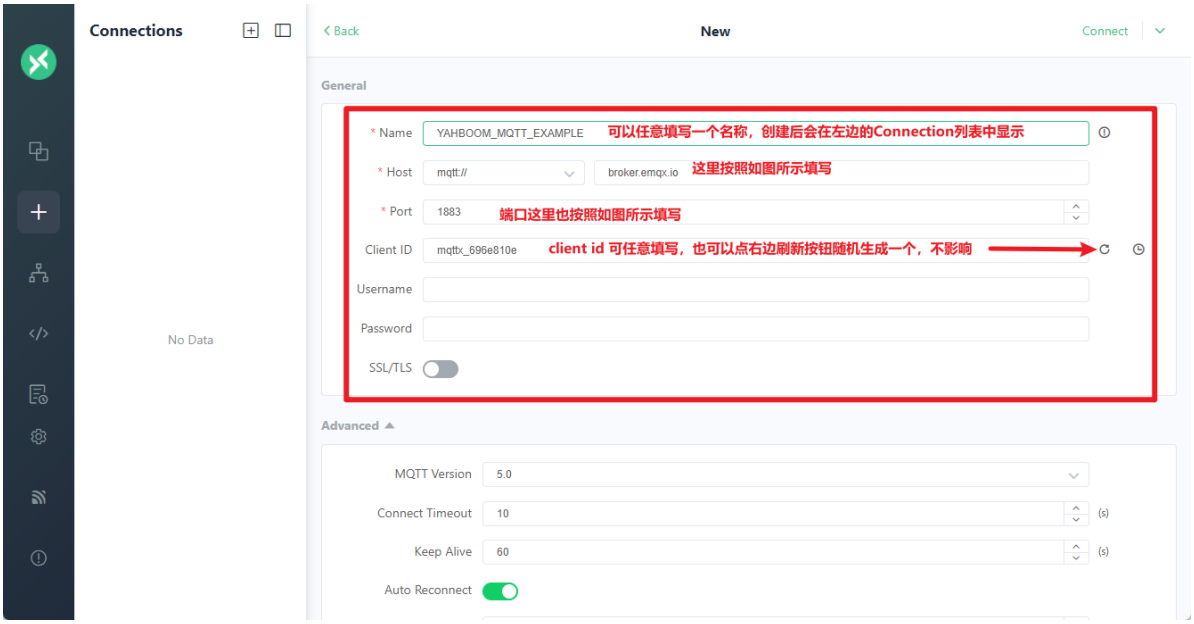


安装步骤比较简单，一直点击 “下一步/next” 即可。

安装完成后我们打开这个程序，点击“New Connection”

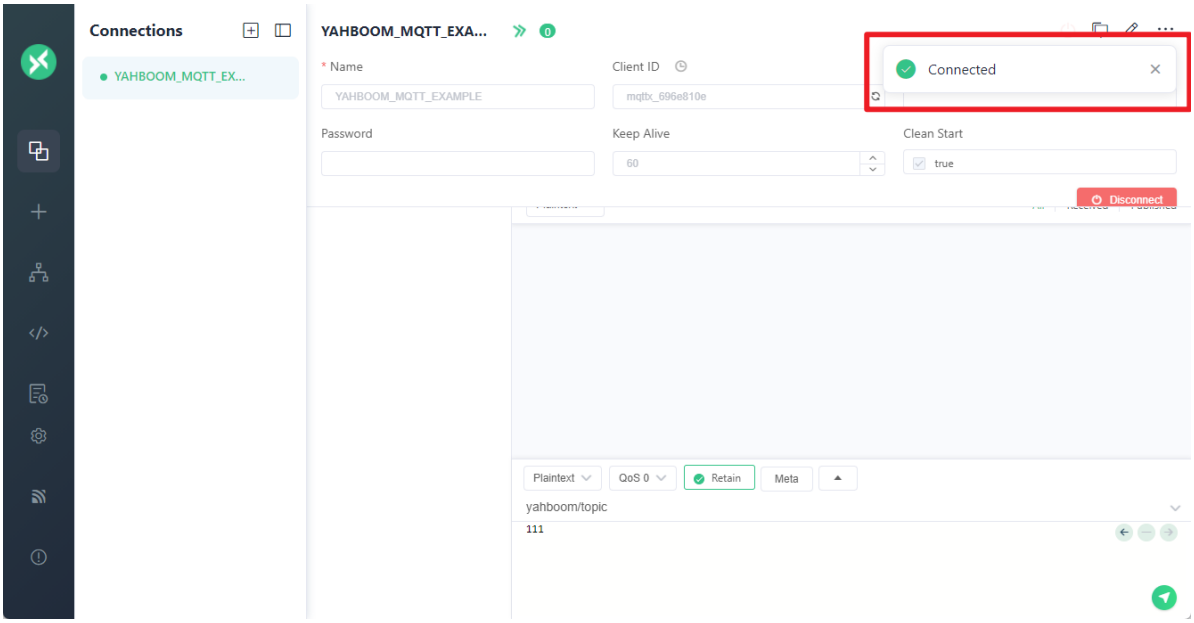


然后填写如下所示的信息

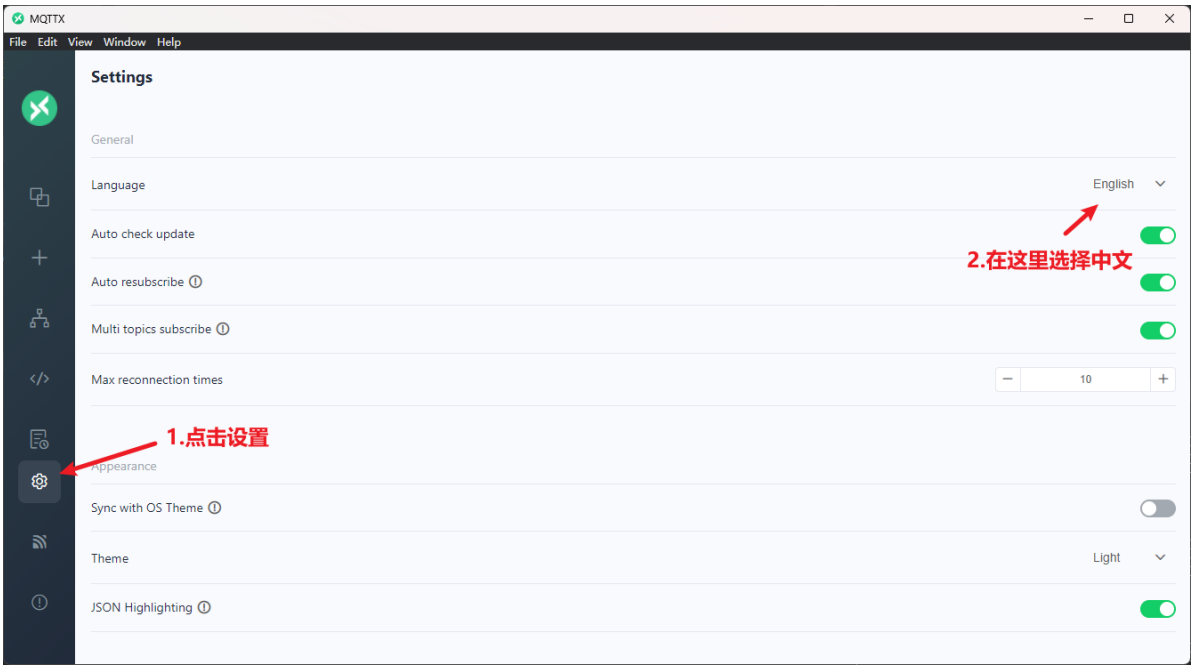


> 这里的Host和Port指的是MQTT服务器的地址信息，我们例程中使用的是EMQX提供的免费的公共MQTT服务器，服务器信息可以在这个链接中找到：
>
> [Free MQTT Broker: Public & Multi-Region | Connect Now | EMQ]
>
> 需要注意的是，第三方服务不能保证长期有效性，如果您发现链接失效，可以及时反馈给我们技术支持，我们会再寻找一个公共MQTT服务器替代并修改教程
>
> 当然，在实际场景（真实业务场景）下，请自行部署MQTT服务器，这样才能保证数据的安全性以及服务的稳定性

填写完成后，点击右上角的Connection，等待弹出Connected则说明连接成功了

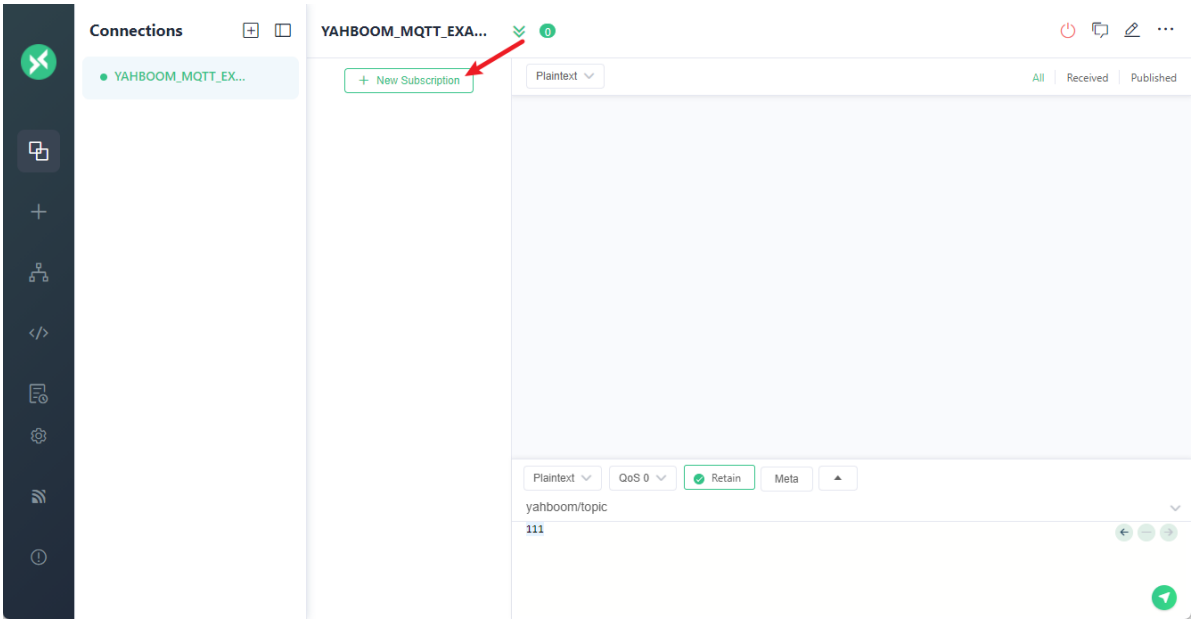


如果需要调整语言，可以在这个地方进行汉化：

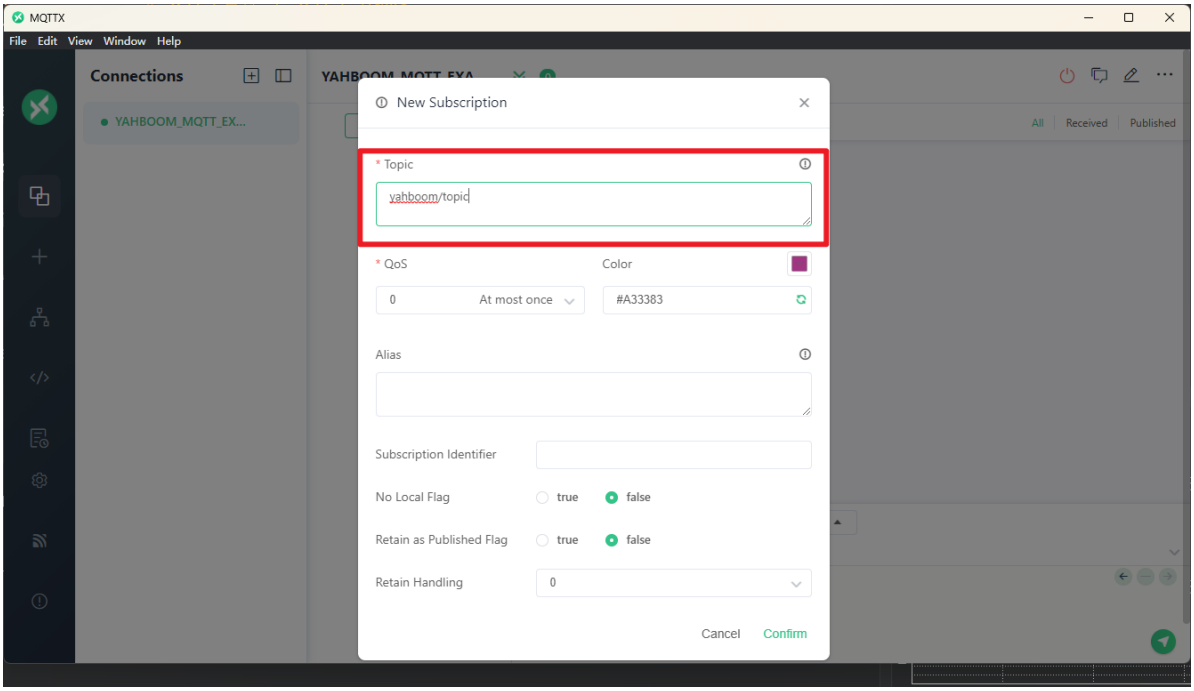


第二步：MQTT-发布者 启动

我们先到MQTTX里，点击New Subscription



填写Topic为yahboom/topic，然后点击右下角Confirm按钮



接着打开本节的示例程序mqtt-publisher，确保K230处在一个能访问互联网的WIFI环境下。

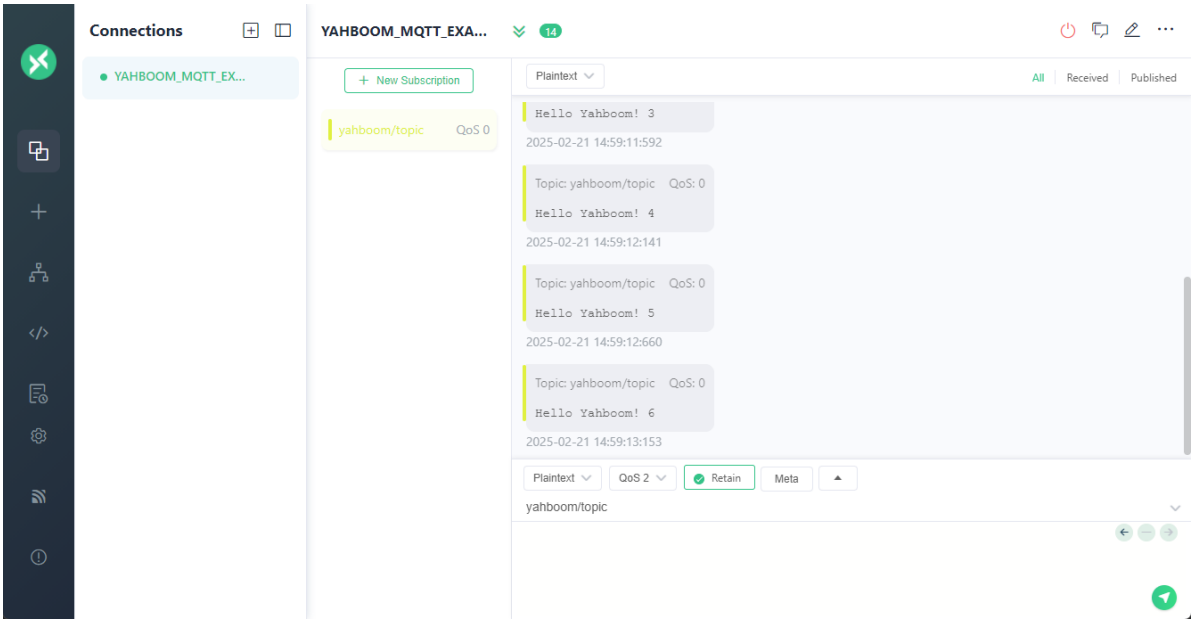
修改WIFI连接SSID和密码

```
# WiFi配置参数 / WiFi configuration parameters
WIFI_SSID = "WIFI SSID"           # WiFi名称 / WiFi name
WIFI_PASSWORD = "WIFI PASSWORD" # WiFi密码 / WiFi password
```

点击运行程序，等待控制台输出

```
连接WIFI: Yahboom.. Connecting..
WIFI连接成功 WIFI Connected!
```

我们回到MQTTX，可以看到这边接收到了K230发布的内容



代码解析

完整代码请参考文件【源码汇总 / 11.Network / 06.mqtt / mqtt_publisher.py】

1. 模块导入部分

代码首先导入了几个关键模块：

- `from mqtt import MQTTClient`：用于实现MQTT通信的核心模块。
- `import network`：用于处理WiFi连接。
- `import os`：提供操作系统相关功能，这里可能用于退出点检查（`os.exitpoint()`）。
- `import machine`：用于硬件控制（尽管本代码中未直接使用）。
- `import time`：用于实现延时操作。

这些模块为后续的WiFi连接和MQTT通信奠定了基础。

2. 配置参数

代码定义了两组全局配置参数：

- **WiFi配置**：
  - `WIFI_SSID = "WIFI SSID"`：WiFi名称。
  - `WIFI_PASSWORD = "WIFI PASSWORD"`：WiFi密码。
  - 这些是占位符，实际使用时需要替换为真实的WiFi凭据。
- **MQTT配置**：
  - `MQTT_BROKER = "broker.emqx.io"`：使用公共的EMQX MQTT服务器。
  - `MQTT_PORT = 1883`：标准的MQTT端口。
  - `MQTT_TOPIC = "yahboom/topic"`：消息发布的主题。

这些参数是程序运行的核心设置。

3. `connect_wifi` 函数

这是一个用于连接WiFi的函数：

- 参数：
  - `ssid`：WiFi名称，默认值为`"Yahboom"`。
  - `password`：WiFi密码，默认值为`"yahboom890729"`。

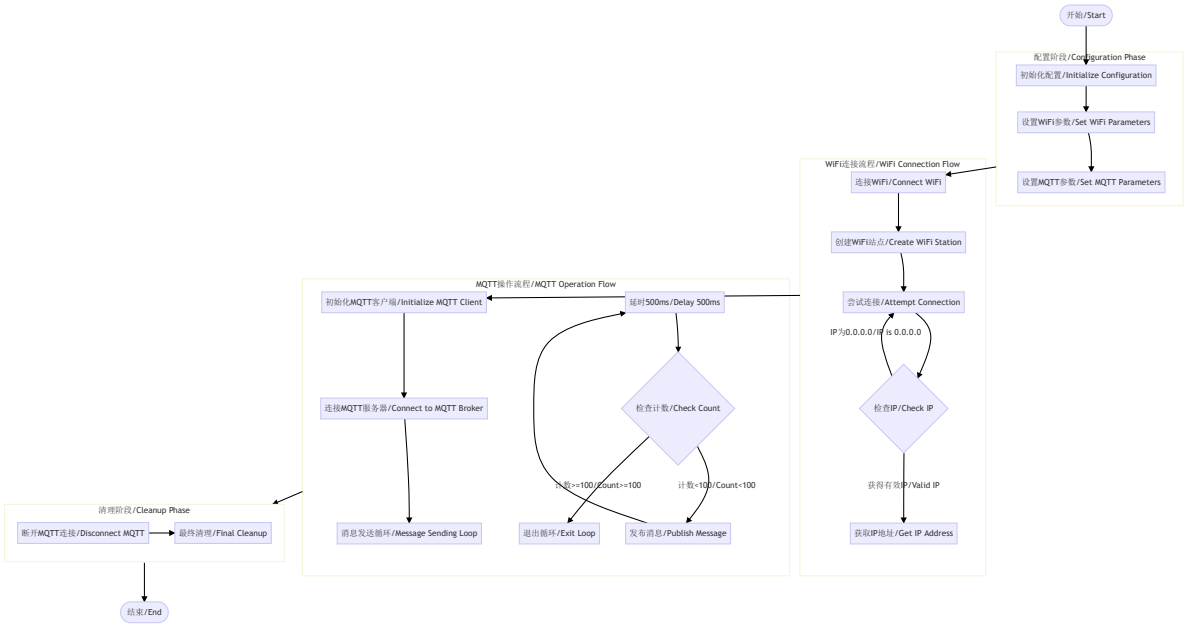
- 功能：
  - 创建一个WiFi站点对象：`wifi_station = network.WLAN(0)`。
  - 调用`connect`方法连接到指定WiFi。
  - 通过循环检查`ifconfig()[0]`（IP地址），直到获取到非`0.0.0.0`的地址，确认连接成功。
  - 使用`os.exitpoint()`，可能是为了处理退出点（在MicroPython中用于检查系统事件）。
- 返回：
  - 返回设备的IP地址：`wifi_station.ifconfig()[0]`。
- 输出：
  - 打印连接状态信息，例如`"连接WIFI: {ssid}.. Connecting.."`和`"WIFI连接成功 WIFI Connected!"`。

4. 主程序部分

主程序在`if __name__ == "__main__":`块中运行：

- **WiFi连接**：
  - 调用`connect_wifi(WIFI_SSID, WIFI_PASSWORD)`连接到WiFi。
- **MQTT客户端初始化**：
  - 创建一个MQTT客户端实例：`client = MQTTClient("YAHBOOM-K230", MQTT_BROKER, port=MQTT_PORT)`。
  - 客户端ID为`"YAHBOOM-K230"`，连接到指定的服务器和端口。
- **连接MQTT服务器**：
  - 调用`client.connect()`建立连接。
- **消息发布循环**：
  - 使用`for`循环发送100条消息。
  - 每次发布消息到`MQTT_TOPIC`，内容为`"Hello Yahboom! " + str(i)`（例如`"Hello Yahboom! 0"`到`"Hello Yahboom! 99"`）。
  - 每次发布后延时500毫秒：`time.sleep_ms(500)`。
- **断开连接**：
  - 循环结束后，调用`client.disconnect()`断开与MQTT服务器的连接。

流程图


开始/Start
配置阶段/Configuration Phase
初始化配置/Initialize Configuration
设置WiFi参数/Set WiFi Parameters
设置MQTT参数/Set MQTT Parameters
WiFi连接流程/WiFi Connection Flow
连接WiFi/Connect WiFi
创建WiFi站点/Create WiFi Station
尝试连接/Attempt Connection
IP为0.0.0.0/IP is 0.0.0.0
检查IP/Check IP
获得有效IP/Valid IP
获取IP地址/Get IP Address
MQTT操作流程/MQTT Operation Flow
初始化MQTT客户端/Initialize MQTT Client
连接MQTT服务器/Connect to MQTT Broker
消息发送循环/Message Sending Loop
延时500ms/Delay 500ms
检查计数/Check Count
计数>=100/Count>=100
计数<100/Count<100
退出循环/Exit Loop
发布消息/Publish Message
清理阶段/Cleanup Phase
断开MQTT连接/Disconnect MQTT
最终清理/Final Cleanup
结束/End